WARSZTAT PRAKTYCZNY

Python w SQL Server:
prognozowanie letniej sprzedaży lodów

Realny scenariusz biznesowy dla sieci lodziarni „Mroźny Kącik". Od pustej instancji, przez analizę danych i trening modelu w Pythonie uruchamianym przez `sp_execute_external_script`, aż po procedurę, która co rano wylicza prognozę i liczbę kuwet lodów do przygotowania.

Parametr	Wartość
Szkolenie	Użycie AI w pracy z SQL Server
Grupa docelowa	Zaawansowani użytkownicy SQL / T-SQL; podstawy Pythona mile widziane, ale niewymagane
Środowisko	SQL Server 2025 Developer Edition (działa też na 2022), SSMS, Machine Learning Services + Python 3.10
Baza	`LodyPrognoza` – tworzona od zera skryptem z Modułu 1 (dane fikcyjne, deterministyczne)
Struktura	4 moduły; każdy kończy się skryptem weryfikacyjnym ze statusem OK / BŁĄD

Spis treści

Spis treści 2
0. Informacje 3
Cele dydaktyczne 3
Scenariusz biznesowy 3
Jak działa sp_execute_external_script 3
Model danych warsztatu 4
Moduł 1. Środowisko i dane od zera 5
Ćwiczenie 1.1 – utworzenie bazy i wygenerowanie danych 5
Ćwiczenie 1.2 – pierwszy skrypt Pythona w SQL Server 5
Weryfikacja Modułu 1: 6
Moduł 2. Eksploracja danych w Pythonie 7
Ćwiczenie 2.1 – dane w obie strony 7
Ćwiczenie 2.2 – wpływ pogody i kalendarza na punkty 7
Ćwiczenie 2.3 – próg opłacalności temperatury dla Plaży 8
Weryfikacja Modułu 2 8
Moduł 3. Trening, porównanie i zapis modelu 9
Wprowadzenie 9
Ćwiczenie 3.1 – tabele modeli i procedura treningowa 9
Ćwiczenie 3.2 – trening 9
Weryfikacja Modułu 10
Moduł 4. Prognoza i decyzja biznesowa 11
Ćwiczenie 4.1 – procedura prognozująca 11
Ćwiczenie 4.2 – prognoza na 1–14 lipca 2026 11
Ćwiczenie 4.3 – od prognozy do decyzji: kuwety 12
Weryfikacja Modułu 4 12
Podsumowanie 13
Porównanie z „ukrytą prawdą” 13
Zadania dodatkowe dla chętnych 13
Załącznik A. Instalacja Pythona dla SQL Server 2022/2025 14
Załącznik B. Sprzątanie po warsztacie 15
Źródła 15

0. Informacje

Cele dydaktyczne

Po warsztacie uczestnik:

- uruchamia kod Pythona wewnątrz SQL Server i rozumie przepływ danych `InputDataSet` → Python → `OutputDataSet`,
- przekazuje parametry i obiekty binarne (model) między T-SQL a Pythonem przez `@params` i parametry OUTPUT,
- przeprowadza eksplorację danych i ocenia jakość danych (braki pomiarów) bez wyprowadzania danych poza serwer,
- trenuje i porównuje modele (prognoza naiwna, regresja liniowa, las losowy) z poprawnym podziałem czasowym,
- przechowuje model w tabeli i wdraża go jako procedurę składowaną zapisującą prognozy,
- krytycznie weryfikuje kod generowany przez asystenta AI w kontekście ML Services (wersje bibliotek, wyciek danych, spójność cech).

Scenariusz biznesowy

Sieć „Mroźny Kąsek" prowadzi cztery punkty sprzedaży lodów rzemieślniczych: kiosk na **Plaży**, ogródek na **Rynku**, budkę w **Parku** oraz wyspę w **Galerii** handlowej. Właścicielka ma dwa powracające problemy: w upalne weekendy lodów brakuje już po południu, a po deszczowych dniach trzeba utylizować produkt, który stracił jakość. Lody są produkowane dzień wcześniej, więc decyzję o wielkości produkcji trzeba podjąć z wyprzedzeniem.

Firma od trzech sezonów (2023–2025) zbiera w SQL Server dzienną sprzedaż w porcjach, ma historię pogody i kalendarz (weekendy, święta, wakacje). Od dostawcy pogody otrzymuje 14-dniową prognozę. **Zadanie zespołu**: zbudować w bazie mechanizm, który na podstawie prognozy pogody i planu promocji przewidzi sprzedaż na 1–14 lipca 2026 r. w każdym punkcie i przeliczy ją na liczbę kuwet do przygotowania.

Dlaczego Python w SQL Server, a nie w osobnej aplikacji? Dane nie opuszczają serwera (bezpieczeństwo, RODO), model jest przechowywany i wersjonowany obok danych, a prognozę uruchamia zwykłe `EXEC` – np. z zadania SQL Server Agent albo z raportu.

Jak działa sp_execute_external_script

SQL Server nie wykonuje Pythona w procesie silnika. Wywołanie `sys.sp_execute_external_script` przekazuje skrypt do usługi **Launchpad**, która uruchamia odizolowany proces Pythona i przesyła do niego dane strumieniowo.

Element	Znaczenie
`@language = N'Python'`	Język skryptu (runtime zarejestrowany w instancji).
`@script`	Kod Pythona jako tekst nvarchar. Wcięcia są znaczące, a apostrofy trzeba podwajać – dlatego w skryptach używamy wyłącznie cudzysłowów "…".
`@input_data_1`	Zapytanie T-SQL, którego wynik trafia do Pythona jako pandas DataFrame o nazwie `InputDataSet`.
`OutputDataSet`	DataFrame zwracany do SQL Server jako zestaw wyników (można go przechwycić przez `INSERT … EXEC`).
`WITH RESULT SETS`	Nadaje nazwy i typy kolumnom zwracanego zestawu wyników.
`@params` + parametry	Deklaracje parametrów; zmienne o tych nazwach (bez @) są dostępne w Pythonie. Parametry OUTPUT pozwalają zwrócić np. model jako varbinary(max).
`print()`	Trafia do zakładki Messages w SSMS (STDOUT message(s) from external script).

Model danych warsztatu

Tabela	Zawartość	Tworzona w
dbo.Punkt	4 punkty sprzedaży; CzyZadaszony = 1 tylko dla Galerii	Moduł 1
dbo.Kalendarz	Dni 1.05–30.09 w latach 2023–2026: dzień tygodnia, weekend, święto, wakacje	Moduł 1
dbo.Pogoda	Historyczna temperatura maks. i opady (2023–2025); 3 dni bez pomiaru temperatury	Moduł 1
dbo.Sprzedaz	Dzienna liczba porcji w punkcie + znacznik promocji (1836 wierszy)	Moduł 1
dbo.PrognozaPogody	Prognoza na 1–14.07.2026 (fala upałów, potem 2 deszczowe dni)	Moduł 1
dbo.PlanPromocji	Zaplanowane promocje w lipcu 2026	Moduł 1
dbo.AnalizaPunktow, dbo.AnalizaTemperatury	Wyniki eksploracji z Pythona	Moduł 2
dbo.Model, dbo.ModelOcena	Zserializowane modele i metryki porównawcze	Moduł 3
dbo.PrognozaSprzedazy	Prognozy dzienne per punkt z identyfikatorem modelu	Moduł 4

Ważne: Dane są generowane deterministycznie (funkcja dbo.fn_Los oparta na MD5), więc każdy uczestnik ma identyczny zbiór, a wartości kontrolne w skryptach weryfikacyjnych są wspólne dla całej grupy. Sprzedaż jest symulowana według ukrytej „prawdy" (np. Plaża silnie zależy od temperatury i deszczu, Galeria prawie wcale) – w Module 2 uczestnicy odkrywają te zależności z danych, a na koniec mogą zajrzeć do skryptu i porównać wnioski z rzeczywistymi parametrami.

Środowisko (Machine Learning Services, Python 3.10, scikit-learn) należy przygotować **przed zajęciami** – instrukcja w Załączniku A.

Moduł 1. Środowisko i dane od zera

Cel modułu
Utworzyć bazę LodyPrognoza z kompletem danych, uruchomić pierwszy skrypt Pythona w SQL Server i potwierdzić, że wymagane biblioteki są dostępne.

Ćwiczenie 1.1 – utworzenie bazy i wygenerowanie danych

Otwórz nowe okno zapytania w SSMS, wklej poniższy skrypt i uruchom go w całości (F5). Skrypt usuwa bazę `LodyPrognoza`, jeśli istnieje, więc w każdej chwili można zacząć warsztat od nowa.

Skrypt: m1_setup.sql

Na co zwrócić uwagę:

- `dbo.fn_Los` daje „losowość", która jest powtarzalna: ten sam klucz (np. `T1` + data) zawsze zwraca tę samą liczbę. Dzięki temu wszyscy mają identyczne dane, a weryfikacja może podać konkretne wartości.
- Dzień tygodnia liczymy od 1900-01-01 (poniedziałek), a nie przez `DATEPART(weekday, …)`, które zależy od ustawienia `SET DATEFIRST` sesji.
- Celowo wprowadzamy 3 dni bez pomiaru temperatury – to realny problem jakości danych, który obsłużymy w Pythonie.
- Tabela `Parametry` w kroku 5 to „ukryta prawda". Na razie jej nie analizujemy – wrócimy do niej w podsumowaniu.

Ćwiczenie 1.2 – pierwszy skrypt Pythona w SQL Server

Uruchom skrypt, który wypisze komunikat w zakładce Messages i zwróci tabelę z wersjami pakietów.

Skrypt: m1_python_test.sql

Oczekiwany wynik (wersje mogą się nieznacznie różnić, istotne jest, że zapytanie zwraca 4 wiersze):

Pakiet	Wersja (przykład)
python	3.10.11
pandas	2.03
numpy	1.26.4
scikit-learn	1.4.2

Zwróć uwagę, że w środowisku SQL Server działa **starsza wersja pandas (1.3.5)** narzucona przez pakiet `revoscalepy`. Część składni z aktualnej dokumentacji pandas 2.x nie zadziała – to częste źródło błędów, także w kodzie z czatów AI.

> **Informacje: gdy kod pisze asystent AI**
>
> Przykładowy prompt: „Napisz wywołanie sp_execute_external_script, które w Pythonie zwróci wersje pandas i numpy jako tabelę".
>
> Typowe problemy w odpowiedziach modeli, które warto wychwycić:
>
> - `@language = N'Python3'` lub `N'py'` – poprawna wartość to `N'Python'`;
> - apostrofy w kodzie Pythona (`'pandas'`) bez podwojenia – psują literał T-SQL;
> - wcięcie całego skryptu Pythona spacjami „dla czytelności" – kończy się `IndentationError`;
> - brak `WITH RESULT SETS` i zaskoczenie nazwami kolumn bez nazw w wyniku;
> - sugestia `pip install` z poziomu skryptu – proces Pythona działa w izolacji, bez uprawnień do instalacji.

Weryfikacja Modułu 1:

Skrypt: m1_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 11 testów powinno mieć status **OK**, a ostatni zestaw wyników: „Moduł 1 zaliczony – możesz przejść dalej”.

- Test 1 = BŁĄD → nie włączono `external scripts enabled` lub nie zrestartowano usługi (Załącznik A, kroki 7–8).
- Błąd Msg 39012/39021 podczas testu Pythona → problem z Launchpad lub uprawnieniami (tabela „Możliwe problemy techniczne”).
- Testy 3–8 = BŁĄD → skrypt 1.1 nie wykonał się do końca; uruchom go ponownie w całości.

Moduł 2. Eksploracja danych w Pythonie

Cel modułu
Przekazać dane do Pythona, policzyć statystyki i korelacje, wykryć braki danych i zapisać wyniki analizy z powrotem do tabel SQL Server.

Ćwiczenie 2.1 – dane w obie strony

Zapytanie z `@input_data_1` staje się DataFrame `InputDataSet`. Skrypt agreguje go w pandas i zwraca jako zestaw wyników.

Skrypt: m2_ex1.sql

Oczekiwany wynik:

Punkt	Dni	Srednia	Minimum	Maksimum
Galeria	459	148,98	79	289
Park	459	160,32	40	437
Plaża	459	230,53	22	844
Rynek	459	249,48	69	733

Pytanie: dlaczego Plaża ma najmniejsze minimum, a jednocześnie największe maksimum? (Odpowiedź: jest najbardziej wrażliwa na pogodę – zobaczymy to w ćwiczeniu 2.2.) W zakładce Messages pokaż też `df.dtypes` – kolumny `decimal` i `bit` warto jawnie rzutować w zapytaniu (`CAST … AS float/int`), żeby uniknąć niespodzianek z typami.

Ćwiczenie 2.2 – wpływ pogody i kalendarza na punkty

Liczymy dla każdego punktu korelację temperatury ze sprzedażą, średnią sprzedaż w dni suche i deszczowe oraz wskaźnik weekendowy. Wynik zapisujemy do tabeli przez `INSERT … EXEC` – kolejność kolumn DataFrame musi odpowiadać liście kolumn w INSERT.

Skrypt: m2_ex2.sql

Oczekiwany wynik (Messages: „Dni bez pomiaru temperatury: 3” oraz daty 2024-07-17, 2024-07-18, 2025-08-05):

Nazwa	KorelacjaTemp	SredniaSucho	SredniaDeszcz	ZmianaWDeszczProc	WskaznikWeekend
Park	0,73	181,2	92,4	-49,0	1,34
Plaża	0,72	274,9	89,7	-67,4	1,32
Rynek	0,70	280,1	150,7	-46,2	1,33
Galeria	0,44	150,5	149,8	-0,5	1,48

Wnioski biznesowe do wspólnego sformułowania: Galeria jest „bezpieczną przystanią” – deszcz jej nie szkodzi, a weekend działa najmocniej (ludzie idą do centrum handlowego). Na Plaży deszcz obniża sprzedaż o 2/3, więc to tam błędna prognoza kosztuje najwięcej.

Ćwiczenie 2.3 – próg opłacalności temperatury dla Plaży

Grupujemy suche dni bez promocji na Plaży w przedziały temperatur (`pd.cut`). Kategorię konwertujemy na tekst (`astype(str)`), bo typ category nie jest obsługiwany przy zwracaniu danych do SQL Server.

Skrypt: m2_ex3.sql

Oczekiwany wynik:

Lp	Przedzial	LiczbaDni	SredniaPorcji
1	ponizej 18	67	135,8
2	18-22	68	197,2
3	22-26	80	271,2
4	26-30	58	390,4
5	30 i wiecej	19	574,8

Zależność nie jest liniowa: każde +4°C to ok. +40–50% sprzedaży. To ważna wskazówka przed Modułem 3 – model liniowy będzie miał z tym kłopot.

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Mam DataFrame z kolumnami PunktID, TempMax, LiczbaPorcji. Policz korelację dla każdego punktu".

- Model często proponuje `df.groupby("PunktID").corr()` – zwraca macierze korelacji dla wszystkich par kolumn (format trudny do zwrócenia do SQL) zamiast jednej liczby na punkt.
- Rzadko sam zauważa wartości NULL (NaN) w temperaturze. Pandas pominie je w `corr()` po cichu – zawsze sprawdzaj braki jawnie, tak jak w ćw. 2.2.
- Używa API z pandas 2.x (np. `DataFrame.map`, `observed=True` jako domyślne zachowanie) – w ML Services działa pandas 1.3.5. W prompcie podawaj wersje bibliotek.
- Korelacja ≠ przyczynowość: o wpływie weekendu i promocji nie dowiemy się z samej korelacji temperatury.

Weryfikacja Modułu 2

Skrypt: m2_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 9 testów powinno mieć status **OK**.

- Testy 1–6 = BŁĄD → ponownie uruchom ćwiczenie 2.2 (tworzy tabelę od nowa).
- Testy 7–9 = BŁĄD → ponownie uruchom ćwiczenie 2.3; sprawdź, czy filtr w zapytaniu dotyczy PunktID = 1.
- Błąd „Column name or number of supplied values does not match" → zmieniona liczba lub kolejność kolumn w DataFrame.

Moduł 3. Trening, porównanie i zapis modelu

Cel modułu
Zbudować procedurę treningową: uzupełnić braki, przygotować cechy, porównać trzy podejścia na sezonie testowym, wybrać najlepsze i zapisać model w tabeli.

Wprowadzenie

- **Podział czasowy, nie losowy.** Uczymy na sezonach 2023–2024, testujemy na 2025. Losowy `train_test_split` „podejrzałby przyszłość" (wyciek danych) i zawyżył ocenę modelu.
- **Punkt odniesienia.** Model ML ma sens tylko wtedy, gdy wyraźnie bije prostą prognozę naiwną: średnią z historii dla danego punktu i typu dnia.
- **Metryki.** MAE – średni błąd w porcjach (rozumie go kierownik), MAPE – średni błąd procentowy (porównywalny między punktami).
- **Model produkcyjny.** Po wyborze algorytmu trenujemy go ponownie na wszystkich danych (2023–2025) i zapisujemy przez `pickle` do kolumny varbinary(max) razem z listą cech.

Ćwiczenie 3.1 – tabele modeli i procedura treningowa

Skrypt: m3_proc.sql

Omów kluczowe fragmenty: interpolację braków po dacie (`interpolate`), funkcję `przygotuj_cechy` (kodowanie punktu jako 4 kolumn 0/1), zapis listy cech w pakiecie oraz przekazanie modelu do T-SQL przez parametr `@model_bin varbinary(max) OUTPUT`.

Ćwiczenie 3.2 – trening

Skrypt: m3_run.sql

Oczekiwany wynik (wartości orientacyjne – drobne różnice zależą od wersji scikit-learn):

Algorytm	MAE_porcje	MAPE_proc	Decyzja
RandomForest	ok. 25,7	ok. 15,1	<-- zapisany jako model 1
LinearRegression	ok. 39,1	ok. 27,1	
Naiwna (srednia)	ok. 65,7	ok. 48,5	

W Messages: „Dni z brakujaca temperatura: 3", „Wiersze treningowe: 1224 | testowe: 612", rozmiar modelu ok. 2,6 MB. **Dyskusja:** dlaczego las losowy wygrywa z regresją liniową? (Zależność od temperatury jest wykładnicza, a wpływ deszczu różny w każdym punkcie – to interakcje, których prosty model liniowy nie uchwyci bez dodatkowych cech.)

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Napisz kod scikit-learn, który przewidzi dzienną sprzedaż na podstawie pogody i kalendarza i oceni model".

- Niemal zawsze pojawia się `train_test_split(…, shuffle=True)` – dla szeregów czasowych to wyciek danych. Wymagaj podziału po dacie.
- Brak punktu odniesienia: model z MAPE 15% wygląda dobrze tylko wtedy, gdy wiemy, że prognoza naiwna ma 48%.
- MAPE liczona z dzieleniem przez wartości bliskie zera lub zerowe – sprawdź, czy dane mogą zawierać 0.
- `get_dummies` bez zapisanej listy kolumn: przy prognozie dla jednego punktu powstanie inna liczba kolumn niż w treningu.

- Zapis modelu do pliku na dysku serwera (`joblib.dump("C:/…")`) – proces Pythona działa w izolowanym kontenerze AppContainer i zwykle nie ma do tego uprawnień; właściwe miejsce to tabela.

Weryfikacja Modułu

Weryfikacja zawiera test funkcjonalny: model jest wczytywany z tabeli i pytany o dwa skrajne scenariusze dla Plaży.

Skrypt: m3_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 8 testów powinno mieć status **OK**. Test 8 pokazuje np. „589 vs 45 porcji”.

- Test 4 = BŁĄD (wybrano LinearRegression) → sprawdź, czy w funkcji `przygotuj_cechy` nie usunięto kolumn punktów lub temperatury.
- Test 5 = BŁĄD (MAPE > 20%) → najczęściej pomylony podział danych lub błąd w zapytaniu wejściowym (np. brak JOIN z Kalendarzem).
- Testy 7–8 = BŁĄD z komunikatem KeyError → lista cech w scenariuszu różni się od listy zapisanej w modelu.
- Każde uruchomienie `usp_TrenujModel` dodaje nowy wiersz w `dbo.Model` – to celowe (wersjonowanie modeli).

Moduł 4. Prognoza i decyzja biznesowa

Cel modułu
Wdrożyć model jako procedurę składowaną, wyliczyć prognozę na 1–14 lipca 2026 i przełożyć ją na liczbę kuwet lodów do przygotowania.

Ćwiczenie 4.1 – procedura prognozująca

Procedura pobiera najnowszy model, sprawdza kompletność prognozy pogody, buduje dane wejściowe (dzień × punkt + plan promocji), uruchamia scoring i w transakcji podmienia prognozy z zakresu dat. Dzięki temu można ją bezpiecznie uruchamiać wielokrotnie, np. codziennie o 6:00 z SQL Server Agent.

Skrypt: m4_proc.sql

Uwaga dydaktyczna: funkcja `przygotuj_cechy` jest skopiowana z treningu. To świadome uproszczenie – w produkcji kod cech powinien być w jednym miejscu (np. własny pakiet Pythona zainstalowany na serwerze), bo rozjazd cech między treningiem a prognozą to jeden z najczęstszych błędów wdrożeń ML. Zabezpieczeniem minimum jest wybór kolumn według listy `pakiet["cechy"]`.

Ćwiczenie 4.2 – prognoza na 1–14 lipca 2026

Skrypt: m4_run.sql

Oczekiwany wynik (orientacyjnie, ±5% w zależności od wersji scikit-learn):

Data	Dzień	Temp	Opady	Plaża	Rynek	Park	Galeria	Razem
2026-07-01	śr	24,5	0,0	281	284	182	129	876
2026-07-02	czw	26,0	0,0	317	293	185	149	944
2026-07-03	pt	28,5	0,0	415	350	234	160	1 159
2026-07-04	sob	31,0	0,0	619	559	348	245	1 771
2026-07-05	nd	32,5	0,0	636	584	348	246	1 814
2026-07-06	pon	29,0	2,5	407	346	236	154	1 143
2026-07-07	wt	21,0	12,0	94	146	100	136	476
2026-07-08	śr	18,5	8,0	61	121	72	125	379
2026-07-09	czw	20,0	0,5	206	231	146	132	715
2026-07-10	pt	23,5	0,0	280	272	169	140	861
2026-07-11	sob	25,0	0,0	382	371	264	224	1 241
2026-07-12	nd	27,0	0,0	456	448	281	218	1 403
2026-07-13	pon	26,5	0,0	374	346	212	151	1 083
2026-07-14	wt	22,0	3,0	223	234	151	132	740

Pytania: Co dzieje się 7–8 lipca? (Plaża spada z ponad 600 do 60–90 porcji, Galeria prawie się nie zmienia.) Czy promocja 4.07 jest widoczna? (Porównaj sobotę 4.07 z niedzielą 5.07 – mimo niższej temperatury sprzedaż na Plaży i Rynku jest zbliżona.)

Ćwiczenie 4.3 – od prognozy do decyzji: kuwety

Prognoza sama w sobie nie jest decyzją. Przeliczamy porcje na kuwety (fikcyjne założenie: 1 kuweta 5 l ≈ 40 porcji) i dodajemy bufor bezpieczeństwa równy błędowi modelu (MAPE) zaokrąglonemu w górę.

Skrypt: m4_business.sql

Przy MAPE ok. 15,1% bufor wynosi 16%. Na upalny weekend Plaża potrzebuje ok. 18–19 kuwet dziennie. **Dyskusja:** czy jeden bufor dla wszystkich punktów ma sens? (Koszt braku towaru na Plaży w upał jest wyższy niż koszt nadwyżki w Galerii – to decyzja biznesowa, nie statystyczna.)

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Napisz procedurę, która wczyta model z tabeli i zapisze prognozy".

- Dynamiczny SQL sklejany z parametrem tekstowym (`@Punkt nvarchar`) bez walidacji – ryzyko SQL injection. U nas sklejamy wyłącznie wartości typu date przekształcone przez `CONVERT(char(8), …, 112)`.
- `INSERT` bez wcześniejszego `DELETE` – ponowne uruchomienie kończy się naruszeniem klucza głównego lub duplikatami.
- Brak walidacji wejścia: prognoza dla dni bez prognozy pogody zwróci puste wyniki „po cichu".
- Prognozy ujemne lub ułamkowe – model regresyjny nie wie, że porcji nie może być -3,7.
- Brak `ModelID` w tabeli prognoz – po kilku treningach nie da się ustalić, który model wygenerował daną liczbę.

Weryfikacja Modułu 4

Skrypt: m4_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 8 testów powinno mieć status **OK**, a podsumowanie: „Moduł 4 zaliczony – warsztat ukończony!".

- Test 1 = BŁĄD (np. 112 wierszy) → niemożliwe przy kluczu głównym; jeśli 0 – procedura zakończyła się błędem, sprawdź Messages.
- Test 3 = BŁĄD → po prognozie ponownie wytrenowano model; uruchom ponownie ćwiczenie 4.2.
- Testy 6–8 = BŁĄD → prognoza nie reaguje na pogodę; sprawdź, czy zapytanie wejściowe pobiera dane z `dbo.PrognozaPogody`, a nie z historycznej `dbo.Pogoda`.

Podsumowanie

Porównanie z „ukrytą prawdą”

Zobacz tabelę `Parametry` ze skryptu Modułu 1 i porównaj z wnioskami z Modułu 2:

Punkt	Wrażliwość na temperaturę	Mnożnik przy ulewie	Mnożnik weekendowy	Wniosek z danych (Moduł 2)
Plaża	0,070 (najwyższa)	× 0,35	× 1,35	deszcz -67%, korelacja 0,72
Rynek	0,050	× 0,55	× 1,35	deszcz -46%
Park	0,055	× 0,55	× 1,35	deszcz -49%, korelacja 0,73
Galeria	0,020 (najniższa)	× 1,05	× 1,50	deszcz ≈ 0%, weekend 1,48

Morał: dane potrafią „odkryć” mechanizm, ale korelacja Plaży (0,72) jest niższa niż wynikałoby z jej wrażliwości, bo część zmienności tłumaczy deszcz. Analizy jednowymiarowe trzeba czytać ostrożnie.

Zadania dodatkowe dla chętnych

- Dodaj cechę `TempMax²` lub logarytmuj sprzedaż i sprawdź, czy regresja liniowa zbliży się do lasu losowego.
- Dodaj trend roczny (np. +5% rok do roku) do generatora danych i sprawdź, jak radzi sobie las losowy, który nie ekstrapoluje.
- Zbuduj procedurę `usp_OcenPrognozy`, która porówna prognozy z rzeczywistą sprzedażą i zapisze MAPE w tabeli monitoringu.
- Zaplanuj zadanie SQL Server Agent uruchamiające `usp_PrognozujSprzedaz` codziennie na kolejne 7 dni.

Załącznik A. Instalacja Pythona dla SQL Server 2022/2025

Od SQL Server 2022 instalator **nie zawiera** już runtime'ów Pythona i R – trzeba je doinstalować samodzielnie. Poniższa procedura pochodzi z dokumentacji Microsoft Learn dla SQL Server 2022 („applies to SQL Server 2022 and later"). Dla SQL Server 2025 społeczność potwierdza jej działanie, ale **przed zajęciami sprawdź aktualną wersję dokumentacji** – ta część produktu zmienia się między wersjami i aktualizacjami CU. Polecenia wykonuj w wierszu poleceń uruchomionym jako administrator; przykłady dotyczą instancji domyślnej MSSQLSERVER.

Krok 1. Funkcja SQL Server

W instalatorze SQL Server (nowa instalacja lub „add features") zaznacz **Database Engine Services** oraz **Machine Learning Services and Language Extensions**.

Krok 2. Python 3.10

Pobierz Pythona 3.10 dla Windows (https://aka.ms/Python3_10), wybierz „Customize installation", zaznacz instalację dla wszystkich użytkowników i ścieżkę `C:\Program Files\Python310`. Nowsze wersje Pythona (np. 3.14) według zgłoszeń użytkowników nie działają z `revoscalepy`.

Krok 3. Plik ograniczeń wersji

Zapisz jako `C:\Program Files\Python310\constraints.txt`:

```
dill==0.3.4
numpy==1.22.0
pandas==1.3.5
patsy==0.5.2
python-dateutil==2.8.2
```

Krok 4. Pakiety bazowe i revoscalepy

```
cd "C:\Program Files\Python310\"
python -m pip install -t "C:\Program Files\Python310\Lib\site-packages" -c "C:\Program
Files\Python310\constraints.txt" dill numpy pandas patsy python-dateutil packaging
https://aka.ms/sqlml/python3.10/windows/revoscalepy-10.0.1-py3-none-any.whl
```

Pakiet `packaging` jest kluczowy – jego brak powoduje na SQL Server 2025 błąd Msg 39012 z komunikatem `Permission denied: 'nul'`.

Krok 5. scikit-learn (wymagany w Modułach 1, 3 i 4)

Wersja musi być zgodna z numpy 1.22. Proponowane wersje (zweryfikuj przy instalacji – pip zgłosi konflikt, jeśli wystąpi):

```
python -m pip install -t "C:\Program Files\Python310\Lib\site-packages" -c "C:\Program
Files\Python310\constraints.txt" scikit-learn==1.3.2 scipy==1.11.4
```

Krok 6. Uprawnienia i rejestracja runtime'u

```
icacls "C:\Program Files\Python310\Lib\site-packages" /grant "NT Service\MSSQLLAUNCHPAD":(OI)(CI)RX /T
icacls "C:\Program Files\Python310\Lib\site-packages" /grant *S-1-15-2-1:(OI)(CI)RX /T

cd "C:\Program Files\Python310\Lib\site-packages\revoscalepy\rxLibs"
.\RegisterRext.exe /configure /pythonhome:"C:\Program Files\Python310" /instance:"MSSQLSERVER"
```

Dla instancji nazwanej użyj konta `NT Service\MSSQLLAUNCHPAD$NAZWA` i parametru `/instance:"NAZWA"`. Po każdej instalacji nowych pakietów warto ponowić polecenia `icacls`.

Krok 7. Włączenie skryptów zewnętrznych

```
EXEC sp_configure 'external scripts enabled', 1;
RECONFIGURE WITH OVERRIDE;
```

Krok 8. Restart i test

Zrestartuj usługę SQL Server (restartuje również Launchpad), a następnie uruchom:

```
EXEC sp_execute_external_script @script = N'import sys; print(sys.version)', @language = N'Python';
```

Jeśli w Messages pojawi się wersja 3.10.x, środowisko jest gotowe – przejdź do weryfikacji Modułu 1.

Załącznik B. Sprzątanie po warsztacie

```
/* Sprzątanie po warsztacie */
USE master;
GO
IF DB_ID(N'LodyPrognoza') IS NOT NULL
BEGIN
    ALTER DATABASE LodyPrognoza SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE LodyPrognoza;
END;
GO
```

Źródła

- Microsoft Learn: Install SQL Server 2022 Machine Learning Services (Python and R) on Windows – https://learn.microsoft.com/en-us/sql/machine-learning/install/sql-machine-learning-services-windows-install-sql-2022
- Microsoft Q&A: SQL Server 2025 – where is Python and R language support – https://learn.microsoft.com/en-us/answers/questions/5626588/sql-server-2025-preview-where-is-python-and-r-lang
- Microsoft Q&A: SQL Server 2025 unable to run Python scripts (rozwiązanie z pakietem packaging) – https://learn.microsoft.com/en-us/answers/questions/5635812/sql-server-2025-unable-to-run-python-scripts
- Microsoft Learn: sp_execute_external_script (Transact-SQL) – dokumentacja parametrów procedury
- Materiał złożony w całość przy użyciu Claude AI.